TL;DR - Comparativo Memórias para os Agentes

Análise comparativa das arquiteturas de memória para Camada Agêntica no contexto do iFood, comparando AWS Bedrock, Google Vertex AI, OpenAI Assistants e a plataforma de orquestração Orq.ai. O ponto principal é que, para garantir a **Soberania dos Dados** (DDD) e evitar o **Lock-in** do provedor de IA, é crucial que o iFood mantenha o controle da memória de longo prazo (Semântica/Personalização). A linha clara de decisão é adotar um **Padrão Híbrido**: **Manter o AWS Bedrock como motor principal** (devido à velocidade e ao ecossistema existente), mas **refinar o Adapter para ser agnóstico**, garantindo que a Memória de Longo Prazo seja armazenada em um Vector Database controlado pelo iFood (como Pinecone ou OpenSearch/pgvector), e não nas *Threads* ou *Memory Stores* proprietárias dos provedores, injetando-a via RAG.

Contexto da Análise já envolve o desenho da estrutura base para solução pensada para o iFood.

1. O Posicionamento da Orq.ai (Antes da Tabela)

É importante separar:

- **AWS, Google, OpenAI:** São **Provedores de Infraestrutura e Modelos**. Eles *têm* o motor e o armazenamento.
- **Orq.ai:** É uma **Camada de Gerenciamento (Middleware)**. Ela não "tem" a memória nativa da mesma forma; ela gerencia os prompts e conecta aos modelos.
    - *Fit no iFood:* A Orq.ai compete mais com a **camada de código "Intel/Tools"** que desenhamos (o orquestrador) do que com a AWS/Google (o storage). Se usarmos Orq.ai, ela seria o "Adapter" pronto, mas vocês teriam menos controle sobre a lógica fina de memória procedural e semântica customizada.

---

2. Tabela Comparativa: Estruturas de Memória para Camada Agêntica

Legenda dos Critérios baseada no quadro do iFood:

- **Episódica (Relacionamento):** Capacidade de manter contexto da conversa (Threads, Histórico).
- **Semântica (Personalização):** Capacidade de aprender fatos e preferências (Sou Vegano, Sou Tier 1).
- **Procedural (Tools/MCPs):** Facilidade de conectar ferramentas (Consultar status do pedido).
- **Lock-in (Engenharia):** Quão difícil é migrar dessa estrutura depois.

Critério	AWS (Bedrock Agent)	Google (Vertex AI Agent Engine)	OpenAI (Assistants API)	Orq.ai (Orquestrador)
Filosofia de Memória	**Estruturada:** Separa Session (curta) de Knowledge Base (longa/RAG). Foco em segurança enterprise.	**Vector-First:** "Memory Stores" altamente integrados com busca vetorial e grounding (Google Search).	**"Black Box" Gerenciada:** Threads persistentes geridas pela OpenAI. O "Memory" é automático e opaco.	**Agnóstica:** Não possui "motor de memória" proprietário. Depende do provider ou de Vector DB externo.
Memória Episódica (Relacioname	✅ **Alta:** Mantém sessão	✅ **Alta:** Mantém histórico longo	✅ **Alta (Threads):** Muito fácil de	🔸 **Média:** Logs de interações

nto)	criptografada. Permite *State management* nativo entre turnos.	e permite grandes janelas de contexto (Gemini 1M tokens) para "lembrar" tudo.	usar, mas os dados ficam "presos" na thread da OpenAI.	existem, mas a injeção do histórico no prompt é manual/configurável.
Memória Semântica (Personalização)	✅ **Alta (Via RAG):** Integração nativa com Vector DB (OpenSearch/Aurora) para buscar perfil do usuário.	✅ **Muito Alta:** O conceito de *Grounding* do Google é fortíssimo para cruzar dados corporativos com preferências.	🔸 **Média:** Feature "Memory" (aprender fatos) ainda é limitada para controle fino de enterprise.	❌ **Baixa:** Você precisa construir o RAG por fora e conectar nela.
Conexão com MCPs (Tools)	✅ **Nativa:** Mapeia Lambda functions como Action Groups. Ótimo para o ecossistema AWS existente.	✅ **Forte:** Extensions e Functions integradas ao ecossistema Google Cloud.	✅ **Forte:** Function Calling é estado da arte, mas requer hospedagem externa das funções.	✅ **Alta:** Foca justamente em facilitar a conexão de Tools de múltiplos providers.
Nível de Lock-in (Risco)	⚠️ **Médio/Alto:** A estrutura Agent amarra você ao Bedrock. O Adapter proposto é essencial aqui.	⚠️ **Alto:** O *Agent Builder* é muito opinativo. Migrar de Vertex Search é complexo.	🔴 **Muito Alto:** Se a lógica de estado está na *Thread* da OpenAI, você não consegue exportar o "cérebro" do agente.	🟢 **Baixo:** A promessa é justamente evitar lock-in, permitindo troca de LLM, mas cria dependência da plataforma deles.

---

3. Análise Profunda para o iFood

O Caso Google (Vertex AI Agent Engine)

O link que você enviou sobre o *Memory Bank* do Google revela uma arquitetura poderosa.

- **Vantagem:** Eles tratam a memória não apenas como "texto salvo", mas como dados estruturados que podem ser consultados via SQL ou Vetor. Para o iFood, isso seria incrível para queries complexas como *"Onde esse cliente pediu pizza nos últimos 3 meses e qual foi a avaliação?"*.
- **Risco:** Exige que os dados (Merchant, Consumer) estejam muito próximos do Google Cloud (BigQuery/Vertex Search). Se o Data Lake do iFood já for AWS, a latência e custo de trânsito de dados (egress) inviabilizam.

O Caso OpenAI (Assistants)

- **Vantagem:** Velocidade absurda de prototipagem.
- **Risco:** Para a "Intel" (Camada de Inteligência) do iFood, usar OpenAI puro é arriscado. Se um cliente disser *"Sou alérgico a camarão"*, e isso ficar salvo apenas na "Thread" da OpenAI, o iFood não possui esse dado estruturado para usar no app principal. **Isso viola a premissa de DDD onde o Domínio (Cliente) deve ser dono do dado.**

O Caso Orq.ai

- Serve muito bem como **Interface para Builders**. Eles entregam uma UI bonita para o time de CX criar prompts e testar.
- Porém, para a **Memória Profunda** (Procedural/Semântica automatizada), ela é insuficiente. Vocês ainda precisariam construir o "Cérebro" (o código Python que desenhamos) e usar a Orq.ai apenas como gateway para o LLM.

---

4. Recomendação de Arquitetura (O Veredito)

Dado o desenho do quadro (Intel + Tools + Infra) e a necessidade de não acoplamento:

Mantenha o AWS Bedrock como motor principal (por velocidade e ecossistema), MAS refine o Adapter Python para usar padrões de armazenamento agnósticos.

O design do código deve evoluir para o seguinte **Padrão Híbrido**:

1. **Memória de Curto Prazo (Episódica/Relacionamento):**
   - Deixe a AWS (ou Google) gerenciar a sessão ativa para velocidade.
   - *No final da sessão*, o Summarization Service (do seu quadro) deve extrair os fatos importantes e salvar no **Banco de Dados do iFood** (não na nuvem de IA).
2. **Memória de Longo Prazo (Personalização):**
   - **Não use o armazenamento interno do Agente (Lock-in).**
   - Use um Vector Database gerenciado (ex: Pinecone, Qdrant ou o próprio

OpenSearch/pgvector) que você controla.
    - O seu Adapter injeta essa memória no prompt via RAG, independente se o LLM é Claude (Bedrock) ou Gemini (Google).

Sim, e essa é uma pergunta estratégica. Você está olhando para "Big Clouds" (AWS/Google) e "Orquestradores" (Orq.ai), mas existe uma categoria emergente focada especificamente em **"Memory as a Service"** ou **"State Management Infrastructure"**.

Para o iFood, que busca desacoplamento e soberania sobre o "Cérebro" (Intel), essas soluções podem ser o **Adapter ideal**, pois elas **nascem agnósticas**.

Aqui estão os dois principais players que faltaram no comparativo e que resolvem exatamente o problema de "Estrutura de Memória": **Zep** e **Mem0** (antigo EmbedChain), além da abordagem de Framework com **LangGraph**.

Novos Players no Radar

1. Zep (Long-term Memory for AI)

O Zep é, talvez, a solução que mais se encaixa no seu desenho do quadro ("Intel" separada de "Infra"). Ele não é um LLM; ele é um serviço de memória que fica *ao lado* do LLM.

- **Diferencial:** Ele faz **Extração de Fatos** e **Sumarização** de forma assíncrona e automática. Se o usuário diz "Não gosto de coentro", o Zep extrai isso como um fato sem você ter que criar um prompt específico para isso.
- **Fit iFood:** Ele separa "Session" (Episódica) de "Facts" (Semântica) nativamente.

2. Mem0 (The Memory Layer for AI)

Focado inteiramente em personalização (User Profile). Ele cria um "Grafo de Memória" do usuário.

- **Diferencial:** Otimizado para entender preferências do usuário ao longo do tempo e recuperá-las automaticamente.
- **Fit iFood:** Excelente para a caixa "Personalização/Preferências Captadas" (Merchants/Consumers) do seu desenho.

3. LangGraph (Persistence Layer)

Não é um SaaS, mas um Framework (Python). É a evolução do LangChain.

- **Diferencial:** Controle total do estado (`Checkpointer`). Você define o esquema da

memória no código e salva onde quiser (Postgres, Redis).
- **Fit iFood:** É a escolha "Build" (construir em casa). Dá trabalho, mas o *lock-in* é zero.

Tabela Comparativa Estendida (Specialized Memory Layers)

Critério	Zep (Memory Service)	Mem0 (Personalization Layer)	LangGraph (Framework)
Filosofia	**Camada de Memória Dedicada:** O LLM é efêmero, o Zep é perene. Processamento assíncrono de contexto.	**Perfil Inteligente:** Focado em criar um "Digital Twin" das preferências do usuário.	**Máquina de Estado:** Controle programático do fluxo e da persistência.
Relacionamento (Episódica)	✅ **Excelente:** Gerencia janelas de contexto e sumariza sessões antigas automaticamente para economizar tokens.	🔸 **Média:** Foca mais no que foi *aprendido* do que no diálogo *verbatim*.	✅ **Alta:** Você define exatamente o que é salvo no checkpoint (Postgres/Sqlite).
Personalização (Semântica)	✅ **Automática:** Detecta entidades e intenções. Classifica "User Facts" separadamente.	✅ **Excelente:** Sua especialidade. Ótimo para dizer "Este cliente é Tier 1 e gosta de Pizza".	🔸 **Manual:** Você precisa escrever a lógica de extração e RAG.

Independência (Agnosticismo)	🟢 **Total:** Funciona com OpenAI, Bedrock, Anthropic, etc. É um "Sidecar".	🟢 **Total:** API independente do LLM.	🟢 **Total:** Código Python puro.
Complexidade de Integração	🟡 **Média:** Requer subir um container (Docker) ou usar a Cloud deles.	🟡 **Baixa:** API Simples.	🔴 **Alta:** Exige engenharia pesada de software para manter o estado.

Como isso altera sua Arquitetura (O Código)

Se adotarmos o **Zep** como o "motor da Intel" (ao invés de depender do Bedrock Agent Memory), o código fica muito mais limpo e alinhado com o DDD, pois o Zep já implementa as distinções de memória que você desenhou.

O Adapter agora não precisa "inventar" a lógica de memória, apenas conectar ao serviço.

Análise de Custo-Benefício para o iFood

1. **AWS Bedrock Agent (Caminho Atual):**
    - *Pró:* Já está na AWS, latência mínima, cobrança unificada.
    - *Contra:* A estrutura de memória é "fechada". Difícil migrar os dados de personalização se trocar de cloud.
2. **Zep (Self-Hosted na AWS):**
    - *Pró:* Você sobe o Zep em um container ECS/EKS na AWS. Os dados ficam no SEU Postgres/VectorDB. **Soberania total.**
    - *Contra:* Mais uma peça de infra para o time de DevOps manter (monitorar, escalar).
3. **LangGraph (Code-First):**
    - *Pró:* Flexibilidade infinita.
    - *Contra:* Você terá que escrever a lógica de "Sumarização" e "Extração de Entidades" na mão (chamar LLM para resumir, chamar LLM para extrair fatos).

Isso aumenta o custo de tokens e latência.

Recomendação Final

Para a **POC/MVP**, o caminho da AWS (Bedrock) é o mais rápido.

Para a Arquitetura de Referência (Produção), eu recomendo fortemente avaliar o Zep (Self-hosted) ou o padrão do LangGraph com Postgres.

Motivo: A imagem do quadro destaca muito a "Segmentação" e "Preferências Captadas". Essas são informações de ouro do iFood. Deixá-las opacas dentro de um "Agent" da AWS é arriscado. Ter um serviço onde você pode rodar um SELECT * FROM memory WHERE user_tier = 'Gold' (como o Zep permite via metadados) é muito valioso para analytics posterior.

Deseja que eu exemplifique como seria a estrutura do banco de dados (Schema) se fossemos usar a abordagem "Code-First" (LangGraph/Custom) para garantir que essas preferências fiquem salvas no PostgreSQL do iFood?